**1.List and Explain the Components of assemblies.

*2.Explain the similarities and differences between Interfaces and Abstract classes

**3.Explain foreach loop with example.

*4.Explain flow control and also explain what is break and continue statement?

*5.What are the rules in defining a constructor? Explain static constructor with example.

*6.Explain method overriding with example

**7.Explain the .NET Framework with its main components and advantages.

*8.Explain namespaces and assemblies in .NET with suitable examples and their importance in application development.

9.Discuss advanced class programming in C# with reference to inheritance, interfaces, polymorphism, and abstraction.

10.Compare C# and VB.NET as NET languages and explain how the NET Framework supports multiple languages.

*11.Explain the complete life cycle of a NET application from source code to execution, highlighting the role of CLR and JIT compiler.

12.Give general form of switch statement .explain with example .

13 .Write a program to demonstrate multiple inheritance

1.List and Explain the Components of assemblies.

Assembly is the basic building block of a .NET application. It is a compiled file (`.dll` or `.exe`) that contains code and information required for execution.

Components of an Assembly:

1. **Manifest**
    - It contains information about the assembly.
    - Includes assembly name, version number, culture, and security details.
    - It acts like an identity card of the assembly.

2. **MSIL (Microsoft Intermediate Language) Code**
    - The source code is converted into MSIL by the compiler.
    - This code is later converted into machine code by the CLR during execution.

3. **Metadata**
    - Contains information about classes, methods, properties, and other data types used in the assembly.
    - Helps the CLR understand the structure of the program.

4. **Resources**
    - Stores non-code files such as images, icons, audio files, and strings.
    - These resources are used by the application when needed.

5. **Type Information**
    - Describes all the types (classes, interfaces, structures, etc.) available in the assembly.
    - Enables other assemblies to access and use these types.

2.Explain the similarities and differences between Interfaces and Abstract classes.

Similarities between Interface and Abstract Class

1. Both are used to achieve **abstraction**.
2. Both cannot created objects directly.
3. Both can contain method declarations that must be implemented by derived classes.
4. Both are used to define common behavior for multiple classes.

Differences between Interface and Abstract Class

Interface	Abstract Class
Contains only method declarations (traditionally).	Can contain both abstract and normal methods.
Does not have constructors.	Can have constructors.
Supports multiple inheritance. A class can implement many interfaces.	A class can inherit only one abstract class.
Used when different classes need the same functionality.	Used when classes share common code and behavior.
All members are public by default.	Members can have different access modifiers (public, private, protected).

3.Explain foreach loop with example.

1.The **`foreach` loop** is used to iterate through each element of an array or collection one by one. It automatically accesses every item without using an index.

2 .**Syntax:**

```
foreach (datatype variable in collection)
{
  // Code to execute
}
```

3 .**Example:**

```
using System;

class Program
{
  static void Main()
  {
     string[] fruits = {"Apple", "Mango", "Orange"};

     foreach (string fruit in fruits)
     {
        Console.WriteLine(fruit);
     }
  }
}
```

4**. Advantages of `foreach` Loop:**

1. Easy to write and understand.
2. No need to use index values.
3. Reduces chances of errors.
4. Useful for arrays and collections.

4.Explain flow control and also explain what is break and continue statement?

1.Flow Control

Flow control means controlling the order in which a program runs its statements. It decides what statement will execute next.

For example:

- Execute statements one by one.
- Make decisions using `if` and `switch`.
- Repeat statements using loops like `for` and `while`.

2.`break` Statement

The **`break` statement** is used to stop a loop immediately.

Example:

```
for(int i = 1; i <= 5; i++)
{
  if(i == 3)
     break;
  Console.WriteLine(i);
}
```

3.`continue` Statement

The **`continue` statement** is used to skip one iteration and continue with the next iteration of the loop.

Example:

```
for(int i = 1; i <= 5; i++)
{
  if(i == 3)
     continue;
  Console.WriteLine(i);
```

}

5.What are the rules in defining a constructor? Explain static constructor with example.

Constructor

A **constructor** is a special method that is automatically called when an object of a class is created. It is used to initialize objects.

Rules for Defining a Constructor

1. The constructor name must be the **same as the class name**.
2. A constructor does **not have any return type**, not even `void`.
3. It is called automatically when an object is created.
4. A constructor can have parameters or no parameters.
5. A class can have more than one constructor (constructor overloading).

Static Constructor

A **static constructor** is used to initialize static data of a class. It is executed only **once**, before the first object is created or before any static member is used.

Example:

```
using System;

class Student
{
  static Student()
  {
     Console.WriteLine("Static Constructor Called");
  }

  static void Main()
  {
     Student s1 = new Student();
     Student s2 = new Student();
```

```
  }
}
```

6.Explain method overriding with example

Method Overriding

Method overriding is a feature in which a child class provides a new implementation of a method that is already defined in the parent class.

It is used to achieve **runtime polymorphism**.

Example:

```
using System;

class Animal
{
  public virtual void Sound()
  {
    Console.WriteLine("Animal makes a sound");
  }
}

class Dog : Animal
{
  public override void Sound()
  {
    Console.WriteLine("Dog barks");
  }
}

class Program
{
  static void Main()
  {
    Dog d = new Dog();
    d.Sound();
  }
}
```

7.Explain the .NET Framework with its main components and advantages.

.NET Framework

The **.NET Framework** is a software platform developed by Microsoft for building and running applications such as web applications, desktop applications, and mobile applications.

Main Components of .NET Framework

1. CLR (Common Language Runtime)

- It is the execution engine of .NET.
- It manages program execution.
- Provides memory management, security, and exception handling.

2. FCL (Framework Class Library)

- It is a collection of pre-written classes and functions.
- Helps developers perform common tasks such as file handling, database access, and networking.

3. CTS (Common Type System)

- Defines data types that can be used in all .NET languages.
- Ensures compatibility between different .NET languages.

4. CLS (Common Language Specification)

- Provides a set of rules that all .NET languages must follow.
- Helps different languages work together.

Advantages of .NET Framework

1. Easy and fast application development.
2. Supports multiple programming languages like C# and VB.NET.
3. Provides security features.
4. Automatic memory management through CLR.

5. Reusable code through class libraries.

8.Explain namespaces and assemblies in .NET with suitable examples and their importance in application development.

Namespace in .NET

A **namespace** is used to organize classes, interfaces, and other program elements. It helps avoid name conflicts and makes the code easier to manage.

Example:

```
using System;

namespace MyApplication
{
  class Program
  {
     static void Main()
     {
        Console.WriteLine("Hello World");
     }
  }
}
```

Assembly in .NET

An **assembly** is the basic building block of a .NET application. It is a compiled file (`.exe` or `.dll`) that contains code, metadata, and resources required to run the application.

Importance in Application Development

Importance of Namespace

1. Organizes code in a proper way.
2. Prevents name conflicts between classes.
3. Makes large applications easier to manage.

Importance of Assembly

1. Enables code reuse.
2. Provides version control and security.
3. Contains all information needed to run the application.

9.Discuss advanced class programming in C# with reference to inheritance, interfaces, polymorphism, and abstraction.

1. Inheritance

- Inheritance allows one class to acquire the properties and methods of another class.
- The existing class is called the **base (parent) class**, and the new class is called the **derived (child) class**.
- It promotes code reusability.

Example:

```
class Animal
{
  public void Eat()
  {
     Console.WriteLine("Eating...");
  }
}

class Dog : Animal
{
}
```

2. Interfaces

- An interface contains method declarations without implementation.
- A class that implements an interface must define all its methods.
- It supports multiple inheritance.

Example:

```
interface IShape
{
  void Draw();
}
```

```
class Circle : IShape
{
  public void Draw()
  {
     Console.WriteLine("Drawing Circle");
  }
}
```

---

3. Polymorphism

- Polymorphism means **one method, many forms**.
- The same method can perform different actions.
- It is achieved using method overriding.

Example:

```
class Animal
{
  public virtual void Sound()
  {
     Console.WriteLine("Animal Sound");
  }
}

class Dog : Animal
{
  public override void Sound()
  {
     Console.WriteLine("Dog Barks");
  }
}
```

---

4. Abstraction

abstraction hides internal details and shows only necessary information.
These features make programs flexible, reusable, and easier to maintain.

Example:

```
abstract class Vehicle
{
   public abstract void Start();
}
```

10.Compare C# and VB.NET as NET languages and explain how the NET Framework supports multiple languages.

Comparison between C# and VB.NET

C#	VB.NET
Uses curly braces `{ }` for blocks.	Uses keywords like `End If`, `End Class`.
Case-sensitive language.	Not case-sensitive.
Syntax is similar to C and C++.	Syntax is simple and English-like.
Widely used for web and desktop applications.	Easy for beginners to learn.
Example: `Console.WriteLine("Hello")`	Example: `Console.WriteLine("Hello")`

How .NET Framework Supports Multiple Languages

The **.NET Framework** supports many languages such as **C#, VB.NET, F#, and C++**. It supports multiple languages through:

1. **CLR (Common Language Runtime)**
    - Executes programs written in different .NET languages.
2. **CTS (Common Type System)**
    - Provides common data types for all languages.
3. **CLS (Common Language Specification)**
    - Defines common rules that all .NET languages follow.
4. **MSIL (Microsoft Intermediate Language)**
    - Source code from different languages is compiled into MSIL, which is then converted into machine code by the CLR.

11.Explain the complete life cycle of a NET application from source code to execution, highlighting the role of CLR and JIT compiler.

1. Writing Source Code

- The programmer writes the code in a .NET language such as **C#** or **VB.NET**.

2. Compilation

- The source code is compiled by the language compiler.
- It is converted into **MSIL (Microsoft Intermediate Language)** or **IL code**.
- This code is stored in an **assembly** (`.exe` or `.dll`).

3. Loading by CLR

- When the program starts, the **CLR (Common Language Runtime)** loads the assembly.
- CLR provides memory management, security, and exception handling.

4. JIT Compilation

- The **JIT (Just-In-Time) Compiler** converts the IL code into machine code.
- Machine code is specific to the computer's processor.

5. Execution

- The machine code is executed by the CPU.
- During execution, the CLR manages memory and removes unused objects using garbage collection.

12.give general form of switch statement .explain with example .

Switch Statement

- The **switch statement** is used to choose one block of code from many options based on the value of an expression.
- It is an alternative to multiple `if-else` statements.
- It checks the value of an expression and executes the matching `case`. If no case matches, the `default` block is executed.

Example :-

```
int day = 2;

switch(day)
{
    case 1:
        Console.WriteLine("Monday");
        break;

    case 2:
        Console.WriteLine("Tuesday");
        break;

    default:
        Console.WriteLine("Invalid Day");
        break;
}
```

13.Write a program to demonstrate multiple inheritance

```
using System;

interface IAnimal
{
    void Eat();
}

interface IPet
{
    void Bark();
}

class Dog : IAnimal, IPet
{
    public void Eat()
    {
        Console.WriteLine("Dog is eating");
    }

    public void Bark()
    {
        Console.WriteLine("Dog is barking");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();

        d.Eat();
        d.Bark();
    }
}
```